

# JSCOUP

User Manual

Version 1.0.0 · Zaidon Aljbaae · MIT License

Part 1 — Integration Guide

Part 2 — API Reference

# Integration Guide

How to embed JSCoup into an existing Python project — Flask, FastAPI, Django, or a standalone/serverless function — step by step.

For the full method-by-method reference, see [API REFERENCE.md](#). This guide is about *wiring it in*; that one is about *what each function does*.

## Contents

---

- [1. Install](#)
- [2. The one object you need: JSCoup](#)
- [3. Flask](#)
- [4. FastAPI / Starlette](#)
- [5. Django](#)
- [6. Watching non-HTTP code \(services, background jobs\)](#)
- [7. Serverless \(AWS Lambda / Azure Functions / OCI Functions\)](#)
- [8. Tracing your database](#)
- [9. Turning on the dashboard login](#)
- [10. Multiple services sharing one dashboard](#)
- [11. A minimal "vendor it inside your own project" pattern](#)
- [12. How APIs, parameters and tokens are detected](#)
- [13. Common mistakes](#)

## 1. Install

---

```
pip install jscoup                # core – zero dependencies, SQLite storage
pip install jscoup[flask]         # + Flask adapter
pip install jscoup[fastapi]      # + FastAPI adapter
pip install jscoup[django]       # + Django adapter
pip install jscoup[sql]          # + real SQL storage (Postgres/MySQL/Oracle via SQLAlchemy 2.x)
pip install jscoup[mongo]        # + MongoDB storage
pip install jscoup[crypto]       # + field-level encryption at rest
```

Extras compose: `pip install "jscoup[fastapi,sql,crypto]"`. Requires Python  $\geq$  3.9.

## 2. The one object you need: JSCoup

---

Everything starts with one instance, conventionally named `b1`:

```

from jscoup import JSCoup

bl = JSCoup(
    service_name="orders-api",
    dashboard_username="admin",
    dashboard_password="change-me", # set your own – see §9
)

```

If you skip `dashboard_username` / `dashboard_password`, JSCoup generates a random password and prints it once to your console at startup, rather than leaving the dashboard open with no login at all.

### 3. Flask

---

```

from flask import Flask
from jscoup import JSCoup

app = Flask(__name__)

bl = JSCoup(service_name="orders-api", dashboard_username="admin", dashboard_password="change-me")
bl.install(app) # wraps every route already registered on `app`

if __name__ == "__main__":
    app.run(port=5000)
    # dashboard: http://127.0.0.1:5000/__jscoup

```

Call `bl.install(app)` **after** your blueprints are registered, or call `bl.refresh_targets()` afterward — the dashboard's route list is a snapshot taken at install time, re-scanned automatically before every dashboard request.

### 4. FastAPI / Starlette

---

```

from fastapi import FastAPI
from jscoup import JSCoup

app = FastAPI()

bl = JSCoup(service_name="orders-api", dashboard_username="admin", dashboard_password="change-me")
bl.install(app) # same call – framework is auto-detected from the object

# dashboard: http://127.0.0.1:8000/__jscoup

```

Async routes are captured natively; no extra configuration needed for `async def` endpoints.

## 5. Django

---

Django has no single "app object" the way Flask/FastAPI do, so wiring is two steps:

```
# app/monitoring.py
from jscoup import JSCoup

bl = JSCoup(service_name="orders-api", dashboard_username="admin", dashboard_password="change-me")
jscoup_middleware = bl.install_django()  # returns a middleware class
```

```
# settings.py
MIDDLEWARE = [
    # ... your existing middleware ...
    "app.monitoring.jscoup_middleware",
]
```

The dashboard is then served through Django's own URL dispatcher at `/__jscoup`.

## 6. Watching non-HTTP code (services, background jobs)

---

A route being captured tells you *an endpoint* failed. `watch / watch_module` tell you *which internal function* failed — useful for pure computation, background jobs, or anything not reached directly over HTTP.

**One function:**

```
@bl.watch(kind="service")
def settle_orders(day: str):
    ...
```

**An entire module, automatically, no per-function decorator:**

```
import app.services  # import first
bl.watch_module("app.services", recursive=True)  # then wrap – see §12 for why order matters
```

**One class imported from elsewhere** (`watch_module` only wraps classes/functions *defined* in the module you pass it):

```
from third_party import SomeServiceClass
bl.watch_class(SomeServiceClass)
```

## 7. Serverless (AWS Lambda / Azure Functions / OCI Functions)

---

```

from jscoup import JSCoup

bl = JSCoup(service_name="my-function", storage=...) # see §10 – needs a shared external store

@bl.watch_lambda()
def handler(event, context):
    ...

```

```

@bl.watch_azure_function()
def main(req):
    ...

```

```

@bl.watch_oci_function()
def handler(ctx, data=None):
    ...

```

There's no local disk to run a dashboard from *inside* the function itself — point `storage=` at a shared `SqlStorage` / `MongoStorage` and run the dashboard from any other process pointed at the same database (see §10).

## 8. Tracing your database

---

```
bl.watch_sqlite() # patches the stdlib sqlite3 module globally
```

```

from sqlalchemy import create_engine
engine = create_engine("postgresql+psycopg://...")
bl.watch_sqlalchemy(engine) # traces this one Engine's statements

```

Either call attaches every SQL statement's text, bind parameters, duration and call site to whichever request is active when it runs.

## 9. Turning on the dashboard login

---

```

bl = JSCoup(
    service_name="orders-api",
    dashboard_username="admin",
    dashboard_password="a-real-secret-from-your-env", # never hard-code in source control
    dashboard_allowed_ips=["203.0.113.4", "10.0.0.0/8"], # optional network allow-list
)

```

Pull the password from an environment variable in real deployments:

```
import os
dashboard_password = os.environ["JSCOUP_DASHBOARD_PASSWORD"]
```

## 10. Multiple services sharing one dashboard

Point every service's `storage=` at the same database (optionally with an isolated schema/prefix so it never collides with your app's own tables):

```
from jscoup.storage.sql import SqlStorage

storage = SqlStorage("postgresql+psycopg://user:pass@host/db", schema="jscoup")
bl = JSCoup(service_name="orders-api", storage=storage)
```

Any one of the services — or a separate, dedicated process — can then serve the dashboard and it will show events from all of them.

## 11. A minimal "vendor it inside your own project" pattern

If you'd rather not depend on an external PyPI/index install (air-gapped environments, monorepo policies), vendor the library directly inside your project instead of installing it as an external package:

```
your_project/
├─ vendor/
│   └─ jscoup-1.0.0/      ← copy of this library's `jscoup/` package + pyproject.toml + LICENSE
├─ backend/
│   └─ requirements.txt   ← production deps – does NOT list jscoup
│   └─ requirements-dev.txt ← "-r requirements.txt" + "-e ../vendor/jscoup-1.0.0"
│   └─ app/
```

```
try:
    from jscoup import JSCoup
except ImportError:
    # not installed (e.g. production image that excludes requirements-dev.txt)
    JSCoup = None

bl = JSCoup(service_name="my-app") if JSCoup is not None else None

def install_jscoup(app):
    if bl is not None:
        bl.install(app)
```

This keeps JSCoup a genuinely optional, dev-only dependency: your production Docker image (built from `requirements.txt` alone) never even sees it, while local development gets full capture and a dashboard.

## 12. How APIs, parameters and tokens are detected

`bl.install(app)` inspects every route and records, per API: its parameters (required vs optional, where each goes — path / query / form / body — plus defaults, allowed values and rules), and whether it needs a token. For FastAPI this comes from FastAPI's own dependency tree, so `Depends(get_current_user)` (and anything it depends on) is understood, `OAuth2PasswordRequestForm` shows `username` / `password`, and an `OAuth2PasswordBearer(auto_error=False)` shows as *token optional*. You can override the token flag for one route with `endpoint.__jscoup_requires_auth__ = True/False`.

`grant_type`, `scope`, `client_id`, `client_secret` on a login route. These are not your app's fields: they are the standard OAuth2 password-form fields that FastAPI's `OAuth2PasswordRequestForm` declares (RFC 6749). The form only *requires* `username` and `password`; the other four are optional and ignored unless your code reads them (`form.scope`, ...). JSCoup lists them because the route accepts them, shows FastAPI's own explanation for each, and tucks them under "optional parameters" on the docs page. If you want a login route that advertises only two fields, declare them yourself (`username: str = Form(...)`, `password: str = Form(...)`) instead of `OAuth2PasswordRequestForm`.

**Open and gated APIs.** By default an API must be marked public (Access tab → Public) to be called through the gateway without a gateway token. Set `gateway_open_tokenless_apis=True` to open every API that needs no token of its own automatically; APIs that do need a token stay gated. On an open API, a bearer token you send is forwarded to the API as its own token. Set `gateway_forward_own_tokens=True` to let an API that *needs* a token be called by presenting that API's own token (for example the one its login route returns): log in, paste the token as the Bearer token on the docs page, and call the protected API — the API itself decides whether to accept it.

**The docs "Run" 401.** Published docs call APIs *through the gateway*. Unless an admin marked the API public (Access tab → Public), the gateway wants a gateway token (from `POST /gw/login`) — even for an API that needs no token of its own, like a login route. The docs page tells you which case you are in.

## 13. Common mistakes

| Mistake                                                                                                                                       | What happens                                                                                                   | Fix                                                                                                                 |
|-----------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------|
| Calling <code>watch_module("app.services")</code> <b>after</b> another module already did <code>from app.services import some_function</code> | The other module's copy of <code>some_function</code> is never wrapped — Python bound that name at import time | Call <code>watch_module</code> immediately after importing the module, before anything else imports names out of it |
| Expecting <code>bl.invoke()</code> to run a plain watched Python function remotely                                                            | Refused — only genuine HTTP API targets can be replayed                                                        | Use the dashboard/API only for real endpoints; a watched function is <i>recorded</i> , not remotely invocable       |
| Calling <code>bl.invoke()</code> (sync) from inside an already-running async event loop                                                       | Deadlocks or raises, depending on context                                                                      | Use <code>await bl.ainvoke(...)</code> instead                                                                      |
| Assuming the dashboard's call count reflects only <i>your</i> manual testing                                                                  | Every call your own code makes to a watched function counts too                                                | Scope <code>watch_module</code> / <code>watch_class</code> narrowly if you only want external HTTP traffic counted  |

| Mistake                                                                        | What happens                                                                                 | Fix                                                                                                               |
|--------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------|
| Leaving <code>dashboard_username</code> unset in a shared/deployed environment | A random password is generated and printed to console once — easy to miss in log aggregation | Always pass an explicit <code>dashboard_password</code> from an environment variable in anything beyond local dev |

# API Reference

Every public class and method in JSCoup 1.0.0. For step-by-step wiring instructions, see the [Integration Guide](#).

## Contents

- [JSCoup — the core class](#)
- [Configuration fields](#)
- [Analyzers](#)
- [Storage backends](#)
- [Gateway](#)
- [Top-level module exports](#)

## JSCoup — the core class

```
from jscoup import JSCoup
```

### Constructor

```
JSCoup(service_name="service", config=None, storage=None, analyzers=None, *, dashboard_password=None,
**options)
```

Builds an instance. Every field listed under [Configuration fields](#) is passed as a keyword in `**options`.

### Instrumentation

| Method                    | Signature                                                                                                                                                          | Description                                                                                                                                                                                                                                             |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>watch</code>        | <code>watch(name=None, kind="service", reraise=True, fallback=None, register=True, tags=None, description="", capture_args=True, healthcheck_allowed=False)</code> | Decorator: wraps one function or coroutine so every call is captured (parameters, exceptions, duration, result preview). <code>register=True</code> also publishes it to the dashboard's Live Tester.                                                   |
| <code>service</code>      | <code>service(name=None, **kwargs)</code>                                                                                                                          | Alias of <code>watch</code> with <code>kind="service"</code> .                                                                                                                                                                                          |
| <code>task</code>         | <code>task(name=None, **kwargs)</code>                                                                                                                             | Alias of <code>watch</code> with <code>kind="task"</code> .                                                                                                                                                                                             |
| <code>watch_module</code> | <code>watch_module(module, kind="service", recursive=False, include=None, exclude=None)</code>                                                                     | Wraps every function <i>and</i> class defined directly in <code>module</code> (object or dotted string). <code>recursive=True</code> also walks every submodule of a package. Must run before another module imports names out of <code>module</code> . |

| Method                        | Signature                                                                 | Description                                                                                                                                                      |
|-------------------------------|---------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>watch_class</code>      | <code>watch_class(cls, kind="service", include=None, exclude=None)</code> | Wraps every plain method defined directly on <code>cls</code> . The single-class counterpart of <code>watch_module</code> , for classes imported from elsewhere. |
| <code>watch_sqlite</code>     | <code>watch_sqlite(global_patch=True)</code>                              | Instruments the standard library <code>sqlite3</code> module globally.                                                                                           |
| <code>watch_sqlalchemy</code> | <code>watch_sqlalchemy(engine)</code>                                     | Instruments one specific SQLAlchemy <code>Engine</code> .                                                                                                        |
| <code>record_query</code>     | <code>record_query(sql, params=None, duration_ms=0.0, ...)</code>         | Manually attach a query to the active capture context — for a database driver with no built-in adapter.                                                          |

## Framework installation

| Method                       | Signature                                                 | Description                                                                                                                        |
|------------------------------|-----------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------|
| <code>install</code>         | <code>install(app=None, mount_path=None, **kwargs)</code> | Detects Flask / FastAPI / Starlette / Django from <code>app</code> and wires capture middleware + the dashboard mount in one call. |
| <code>install_django</code>  | <code>install_django(mount_path=None, **kwargs)</code>    | Explicit Django path — returns a middleware class to add to <code>MIDDLEWARE</code> yourself.                                      |
| <code>refresh_targets</code> | <code>refresh_targets() -&gt; int</code>                  | Re-scans the installed app's routes right now. Called automatically before every dashboard request; safe to call any time.         |

## Serverless

| Method                            | Signature                                        | Description                                                                                                                |
|-----------------------------------|--------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| <code>watch_lambda</code>         | <code>watch_lambda(name=None)</code>             | Decorator for a real, deployed AWS Lambda <code>handler(event, context)</code> — every real invocation is captured.        |
| <code>watch_azure_function</code> | <code>watch_azure_function(name=None)</code>     | Same, for an Azure Functions <code>main(req)</code> handler.                                                               |
| <code>watch_oci_function</code>   | <code>watch_oci_function(name=None)</code>       | Same, for an OCI (Oracle Cloud) Functions handler.                                                                         |
| <code>simulate_lambda</code>      | <code>simulate_lambda(handler, name=None)</code> | Registers a Lambda handler with the dashboard's Live Tester for on-demand invocation, separate from watching real traffic. |

## Replay

| Method              | Signature                                                                         | Description                                                                                               |
|---------------------|-----------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| <code>invoke</code> | <code>invoke(target_id, params=None, token=None, base_url=None) -&gt; Dict</code> | Programmatic, synchronous replay of a previously discovered HTTP target. Refuses if called from inside an |

| Method                       | Signature                                                                                | Description                                                                                |
|------------------------------|------------------------------------------------------------------------------------------|--------------------------------------------------------------------------------------------|
|                              |                                                                                          | already-running event loop — use <code>ainvoke</code> there instead.                       |
| <code>ainvoke</code>         | <code>await ainvoke(target_id, params=None, token=None, base_url=None) -&gt; Dict</code> | Async replay — awaits directly on the caller's own event loop, preserving capture context. |
| <code>register_target</code> | <code>register_target(target) -&gt; Target</code>                                        | Manually register a <code>Target</code> the automatic discovery didn't find.               |

## Diagnostics & maintenance

| Method                                    | Signature                                                             | Description                                                                                                                                         |
|-------------------------------------------|-----------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|
| <code>check_apis</code>                   | <code>check_apis(targets=None, timeout=5.0)</code>                    | Sweeps every target that opted in with <code>healthcheck_allowed=True</code> and reports which respond.                                             |
| <code>audit_code</code>                   | <code>audit_code(paths=None)</code>                                   | Static source-code scan (no execution) for common defensive-programming issues.                                                                     |
| <code>identify</code>                     | <code>identify(token=None, **fields)</code>                           | Manually attach an actor to the active capture context.                                                                                             |
| <code>note</code>                         | <code>note(message, category="note", level="info", **data)</code>     | Attach a free-form breadcrumb to the active capture context.                                                                                        |
| <code>metrics</code>                      | <code>metrics() -&gt; dict</code>                                     | Background-delivery counters: <code>before_store_failures</code> , <code>storage_failures</code> , publisher accepted/saved/dropped/failed/pending. |
| <code>flush</code>                        | <code>flush(timeout=5.0) -&gt; bool</code>                            | Wait for queued background work. <code>False</code> means work remains.                                                                             |
| <code>close</code>                        | <code>close(timeout=5.0) -&gt; bool</code>                            | Flush and release every open connection. Call during application shutdown.                                                                          |
| <code>rotate_dashboard_credentials</code> | <code>rotate_dashboard_credentials(password, credential_epoch)</code> | Change the dashboard password and revoke every existing session at once.                                                                            |
| <code>describe</code>                     | <code>describe(...)</code>                                            | Attach a hand-written description/examples to a registered target, for the published docs view.                                                     |

## Class methods (alternate constructors)

| Method                          | Signature                                                                                                     | Description                                                                                                      |
|---------------------------------|---------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------------------------------|
| <code>JSCoup.auto</code>        | <code>auto(app=None, service_name=None, *, dashboard_username=None, dashboard_password=None, **kwargs)</code> | Build + <code>watch_sqlite()</code> + <code>install(app)</code> in one call — the fastest path to full coverage. |
| <code>JSCoup.high_volume</code> | <code>high_volume(service_name="service", **options)</code>                                                   | Pre-configured for high-throughput services: background delivery on, success-capture off.                        |

| Method                               | Signature                                                       | Description                                                                                           |
|--------------------------------------|-----------------------------------------------------------------|-------------------------------------------------------------------------------------------------------|
| <code>JSCoup.from_config_file</code> | <code>from_config_file(app=None, path=None, **overrides)</code> | Build from a JSON project-config file instead of keyword arguments; auto-generates one on first load. |

## Configuration fields

Passed as keyword arguments to `JSCoup(...)`.

### Identity & storage

| Field                                                 | Default                                           | Meaning                                         |
|-------------------------------------------------------|---------------------------------------------------|-------------------------------------------------|
| <code>service_name</code>                             | <code>"service"</code>                            | Label attached to every captured event.         |
| <code>environment</code>                              | <code>\$JSCOUP_ENV</code> or <code>"local"</code> | Free-text environment tag.                      |
| <code>db_path</code>                                  | <code>.jscoup/events.db</code>                    | SQLite file for the default storage backend.    |
| <code>storage</code>                                  | <code>SQLiteStorage</code>                        | Any <code>BaseStorage</code> instance.          |
| <code>max_events</code> / <code>retention_days</code> | <code>5000</code> / <code>14</code>               | Storage caps; oldest rows purged automatically. |

### Dashboard & sessions

| Field                                                                     | Default                           | Meaning                                                                                  |
|---------------------------------------------------------------------------|-----------------------------------|------------------------------------------------------------------------------------------|
| <code>mount_path</code>                                                   | <code>/__jscoup</code>            | Where the dashboard/gateway are mounted.                                                 |
| <code>dashboard_enabled</code>                                            | <code>True</code>                 | Master switch.                                                                           |
| <code>dashboard_mount_in_app</code>                                       | <code>True</code>                 | Mount into the same app/port instead of standalone.                                      |
| <code>dashboard_username</code> / <code>dashboard_password</code>         | <code>None</code>                 | Login credentials (password hashed immediately, PBKDF2-HMAC-SHA256, 600,000 iterations). |
| <code>dashboard_local_dev</code>                                          | <code>False</code>                | Explicit opt-in to run with no login, for throwaway local runs.                          |
| <code>dashboard_session_hours</code>                                      | <code>12.0</code>                 | How long a login session stays valid.                                                    |
| <code>dashboard_login_max_attempts</code> / <code>_lockout_seconds</code> | <code>5</code> / <code>300</code> | Brute-force lockout, keyed by source+account.                                            |
| <code>dashboard_allowed_ips</code>                                        | <code>None</code>                 | IP/CIDR allow-list checked before login runs.                                            |
| <code>dashboard_max_body_bytes</code>                                     | <code>16 MiB</code>               | Hard cap on dashboard request bodies.                                                    |
| <code>trust_proxy_headers</code>                                          | <code>False</code>                | Only enable behind a proxy that strips client-supplied forwarding headers itself.        |

### Route discovery (what `install(app)` learns about each API)

Each discovered HTTP API becomes a `Target` with these fields (also returned by `GET /__jscoup/api/targets`):

| Field         | Meaning                                                                                                                                                                                                       |
|---------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| params        | List of ParamSpec : name , location ( path / query / form / body / argument ), type , required , default , description , choices (allowed values), constraints ( pattern , ge , le , min_length ...), example |
| requires_auth | True when the API needs a token                                                                                                                                                                               |
| auth_optional | True when a token is accepted but not required                                                                                                                                                                |
| auth_scheme   | bearer , basic , apikey:<header> ...                                                                                                                                                                          |
| public        | (docs data only) True when an admin marked the API public through the gateway                                                                                                                                 |

For **FastAPI**, all of this is read from FastAPI's own resolved dependency tree, so it matches what FastAPI itself will accept: inputs declared by `Depends(...)` classes/functions (for example `OAuth2PasswordRequestForm`) are included, and a security scheme anywhere in the dependency chain marks the API as needing a token. Flask and Django use their own detection (see the Integration Guide).

## Gateway

| Field                                | Default                         | Meaning                                                                                                                                                                                                                                                      |
|--------------------------------------|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| gateway_enabled                      | False                           | Turns on <code>POST /gw/login</code> and per-user scoped proxying.                                                                                                                                                                                           |
| gateway_db_path                      | <code>.jscoup/gateway.db</code> | Realm-specific local SQLite file.                                                                                                                                                                                                                            |
| gateway_token_ttl_hours              | 1.0                             | How long a login-issued token stays valid.                                                                                                                                                                                                                   |
| gateway_open_tokenless_apis          | False                           | Treat every API that needs no token of its own as open through the gateway (no gateway token required), as if marked public. APIs that do need a token are never opened by this.                                                                             |
| gateway_forward_own_tokens           | False                           | Let a caller reach an API that needs a token by presenting that API's own token (e.g. from its login route) as the Bearer token; the gateway forwards it and the API accepts or rejects it. A valid gateway login token is still treated as a gateway token. |
| gateway_rate_limit /<br>_rate_window | 120 / 60s                       | Requests allowed per token per window.                                                                                                                                                                                                                       |
| invoke_max_concurrency               | 8                               | Concurrent replay/proxy calls allowed per process.                                                                                                                                                                                                           |
| invoke_max_response_bytes            | 2 MiB                           | Replay responses larger than this return an error.                                                                                                                                                                                                           |
| redact_replay_response               | False                           | Mask token/password-like fields in the response that replay, the Live Tester and gateway calls hand back. Off by default (the response is the API's real answer); captured events, call logs and stored previews are always redacted.                        |

## Capture behaviour

| Field   | Default | Meaning                        |
|---------|---------|--------------------------------|
| enabled | True    | Master switch for all capture. |

| Field                                                                                 | Default                                                  | Meaning                                                                    |
|---------------------------------------------------------------------------------------|----------------------------------------------------------|----------------------------------------------------------------------------|
| <code>capture_success</code>                                                          | <code>False</code>                                       | Also store full detail for successful calls.                               |
| <code>capture_slow</code> / <code>slow_request_ms</code> / <code>slow_query_ms</code> | <code>True</code> / <code>1000</code> / <code>200</code> | Flag slow requests/queries even when they succeed.                         |
| <code>n_plus_one_threshold</code>                                                     | <code>8</code>                                           | Queries-per-request above this trips the N+1 detector.                     |
| <code>capture_headers</code>                                                          | <code>True</code>                                        | Include request headers (after redaction).                                 |
| <code>capture_print</code>                                                            | <code>False</code>                                       | Tee <code>sys.stdout</code> and attach printed output to the active event. |
| <code>sample_rate</code>                                                              | <code>1.0</code>                                         | Head sampling — unsampled calls are not recorded at all.                   |

## Privacy & encryption

| Field                                           | Default                                   | Meaning                                                                      |
|-------------------------------------------------|-------------------------------------------|------------------------------------------------------------------------------|
| <code>sensitive_keys</code> / <code>mask</code> | built-in list / <code>"[redacted]"</code> | Field names redacted from captured payloads.                                 |
| <code>encryption_key</code>                     | <code>None</code>                         | Fernet key ( <code>jscoup[crypto]</code> ) — field-level encryption at rest. |
| <code>before_store</code>                       | <code>None</code>                         | Callback to inspect/rewrite/suppress an event before persistence.            |
| <code>on_event</code>                           | <code>None</code>                         | Callback fired for every fully-stored event.                                 |

## Analizers

`jscoup.analyzers` — a fixed, priority-ordered chain. The first analyzer whose rules match wins:

1. **Database** — missing table/column, unique/FK/NOT NULL/check violations, deadlocks, pool exhaustion, "not found" ORM exceptions.
2. **Data shape** — type mismatches, unexpected `None`, malformed JSON.
3. **Validation** — missing/invalid required fields.
4. **Network** — DNS failures, connection refused/timeouts.
5. **Auth** — missing/expired/invalid tokens, permission decorator rejections.
6. **Access** — 403-shaped authorization failures.
7. **HTTP** — generic 4xx/5xx.
8. **Performance** — slow requests/queries, N+1 patterns.
9. **Config** — missing environment variables, misconfigured connections.
10. **Generic** (fallback) — anything unmatched; still returns the exact source line.

Pass your own via `JSCoup(analyzers=[...])`.

## Storage backends

`jscoup.storage` — all implement the same `BaseStorage` interface.

| Class                      | Backend                                    | Extra                      | Use when                                                                                                    |
|----------------------------|--------------------------------------------|----------------------------|-------------------------------------------------------------------------------------------------------------|
| <code>SQLiteStorage</code> | Local SQLite file                          | none                       | Default — single process, no shared infra.                                                                  |
| <code>SqlStorage</code>    | Postgres / MySQL / Oracle (SQLAlchemy 2.x) | <code>jscoup[sql]</code>   | Multiple workers/replicas sharing one dashboard; supports <code>schema="jscoup"</code> for table isolation. |
| <code>MongoStorage</code>  | MongoDB                                    | <code>jscoup[mongo]</code> | App already standardizes on MongoDB.                                                                        |
| <code>MemoryStorage</code> | In-process dict                            | none                       | Unit tests.                                                                                                 |

## Gateway

`jscoup.gateway.GatewayStore` — admin-curated per-user access to the discovered API surface. Key methods: `create_user(username, password, allowed_target_ids, allowed_ips=None)`, `login(username, password, client_ip)`, `revoke(user_id)`, `authenticate(token)`. See the [Integration Guide](#) and the White Paper for the full trust model.

## Top-level module exports

```
from jscoup import (
    JSCoup, JSCoupConfig, LambdaContext,
    EventRecord, QueryRecord, Breadcrumb, Diagnosis, Actor, build_fingerprint,
    BaseStorage, SQLiteStorage, MemoryStorage,
    AnalyzerEngine, BaseAnalyzer, AnalysisInput,
    DatabaseAnalyzer, DataShapeAnalyzer, ValidationAnalyzer, AuthAnalyzer,
    AccessAnalyzer, NetworkAnalyzer, ConfigAnalyzer, HttpAnalyzer,
    PerformanceAnalyzer, GenericAnalyzer,
    connect, emit_query, instrument_sqlite3, uninstrument_sqlite3,
    instrument_sqlalchemy, uninstrument_sqlalchemy, instrument_django,
    IdentityResolver, fingerprint_token, decode_jwt_payload, Redactor, hash_password,
    Registry, Target, ParamSpec, Simulator,
    ConnectivityCheck, check_connectivity, AuditFinding, audit_code,
    ApiCheckResult, check_apis,
    GatewayStore, GatewayUser, ALL_TARGETS,
    EncryptingStorage, generate_encryption_key,
    run_load_test, format_loadtest_report,
    current_context, all_contexts, add_breadcrumb,
    __version__,
)
```